

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192525199
Name	G. Venkata Niranjan Reddy

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

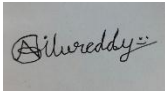
Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I G. Venkata Niranjan Reddy (192525199) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. Hadoop Job Failure Due to Uneven Input Splits and Idle Reducers

Identification of Error

The most likely design error is poor data partitioning and load imbalance in the MapReduce job design. The input data is not being divided into evenly distributed input splits, causing some mapper tasks to process much more data than others. In addition, the partitioning of intermediate key-value pairs may be poorly designed, resulting in some reducers receiving little or no data and remaining idle.

Cause Analysis

In Hadoop MapReduce, the input data is divided into Input Splits, and each split is normally processed by a mapper. If the splits are uneven, some mapper tasks finish quickly while others continue processing large amounts of data. This creates a straggler problem, where the entire job must wait for the slowest mapper.

After the map phase, Hadoop performs the shuffle and sort phase, where intermediate records are distributed to reducers. By default, the partition is often determined using a hash-based mechanism:

Therefore, the main issue is not simply that Hadoop has insufficient nodes. The actual design error is data skew combined with inefficient input splitting and reducer partitioning.

Corrective Actions

The following improvements should be implemented:

1. Use an appropriate input format based on the structure and size of the data.
2. Use Combine File Input Format when processing many small files so that multiple small files can be combined into larger logical splits.
3. Adjust HDFS block size and input split size when required to create better-balanced mapper workloads.
4. Implement a custom partitioner if the default hash partitioning causes skewed reducer workloads.
5. Use key salting for extremely frequent keys. A random or controlled suffix can distribute records with the same popular key across multiple reducers.
6. Select an appropriate number of reducers based on data volume and available cluster resources.

7. Use a combiner where applicable to reduce the amount of intermediate data transferred during the shuffle phase.
8. Monitor mapper and reducer execution times through YARN or Hadoop job history to identify straggler tasks.

Use of Hadoop Concepts

This solution applies the concepts of HDFS blocks, Input Splits, Map tasks, Shuffle and Sort, Partitioner, Combiner, Reducer parallelism, and data skew handling.

2. HDFS Data Unavailability After a Single Node Failure

Identification of Error

The most probable design error is an incorrect HDFS replication configuration or a lack of Name Node High Availability. If data becomes unavailable immediately after one Data Node fails, it strongly suggests that some blocks have an insufficient replication factor or that all replicas are stored in a location vulnerable to the same failure.

Cause Analysis

HDFS achieves fault tolerance by dividing files into blocks and storing multiple copies of each block on different Data Nodes. For example, if the replication factor is 3, each block should normally have three copies.

If the replication factor is incorrectly set to:

then the failure of the Data Node containing that block immediately makes the data unavailable.

Even if the replication factor is greater than 1, poor replica placement can create another problem. For example, if replicas are placed on Data Nodes belonging to the same rack, a rack failure or network problem could affect all copies.

Thus, the root cause may be a combination of low replication, poor rack awareness, and absence of Name Node High Availability.

Corrective Actions

The following corrections are recommended:

1. Configure a replication factor of at least 3 for critical production data.

2. Use rack-aware replica placement so that replicas are distributed across different Data Nodes and racks.
3. Enable Name Node High Availability with an Active Name Node and Standby Name Node.
4. Use Journal Nodes or another supported mechanism to synchronize metadata changes.
5. Configure automatic failover where appropriate.
6. Monitor under-replicated blocks regularly.
7. Use HDFS balancer tools to improve storage and replica distribution.
8. Perform periodic failure testing by intentionally removing a DataNode from service and verifying continued data availability.

Use of Hadoop Concepts

The solution uses HDFS replication, block storage, Data Nodes, Name Node High Availability, Active/Standby architecture, rack awareness, Journal Nodes, and HDFS balancing.

3. Duplicate Records in ETL and Apache NiFi Data Ingestion

Identification of Error

The likely design error is the absence of idempotent ingestion and proper duplicate handling. The workflow may allow the same record to be processed multiple times during retries, failures, restarts, or overlapping ETL jobs.

Apache NiFi provides reliable data flow processing, but if the destination system accepts repeated writes without checking record identity, duplicate records can still occur.

Cause Analysis

Duplicate records may be produced for several reasons:

- A NiFi processor retries data after a temporary failure.
- The record was successfully written to the destination, but the acknowledgement was not received.
- The workflow restarts and processes previously consumed data again.
- Multiple NiFi flows read the same source.

Therefore, assuming that the ingestion pipeline automatically provides exactly-once storage is a major design mistake.

Corrective Actions

A stronger ingestion design should include:

1. Generate a unique identifier for every source record.
2. Use a primary key or unique constraint in the analytics database.
3. Implement idempotent writes, meaning repeated processing of the same record does not create additional records.
4. Store ingestion offsets, checkpoints, or processing state.
5. Configure NiFi retry behavior carefully.
6. Use deduplication logic based on record ID, source ID, timestamp, or a generated hash.
7. Use UPSERT or merge operations instead of unconditional insert operations where supported.
8. Ensure that multiple ETL jobs do not process the same time range.
9. Maintain a dead-letter or failure queue for records that cannot be processed successfully.

Use of Hadoop and Integration Concepts

This solution uses concepts of Apache NiFi data flow, ETL, idempotency, at-least-once delivery, unique keys, checkpoints, offsets, retries, deduplication, and UPSERT operations.

The main objective is to make the pipeline reliable even when processors fail or data is replayed.

4. YARN Resource Starvation with Multiple Concurrent Users

Identification of Error

The primary design error is improper YARN resource scheduling and the absence of effective queue-based resource control. Multiple users are submitting jobs into the cluster, but resource allocation is not controlled according to workload priority, user limits, or queue capacity.

Cause Analysis

YARN separates cluster management from application processing. The Resource Manager controls global resource allocation, while Node Managers manage resources on individual nodes.

If all users submit jobs into one common queue without restrictions, a large application can request many containers and consume most of the available resources.

For example:

If several large jobs consume nearly all available containers, new jobs cannot obtain sufficient memory or CPU. This produces resource starvation.

Corrective Actions

A better YARN resource management strategy should include:

1. Use the Capacity Scheduler to allocate fixed or guaranteed percentages of cluster resources to different queues.
2. Alternatively, use the Fair Scheduler when fair sharing among users and applications is required.
3. Create separate queues for ETL, reporting, machine learning, and high-priority workloads.
4. Configure minimum and maximum resource capacity for each queue.
5. Apply user-level limits to prevent one user from monopolizing resources.
6. Set application priorities where supported.
7. Enable preemption for critical workloads if appropriate.
8. Configure realistic memory and virtual CPU requirements for containers.
9. Monitor cluster utilization and identify applications with excessive resource requests.
10. Reserve sufficient resources for critical or time-sensitive applications.

Example Queue Design

A possible configuration could be:

Queue	Workload	Resource Allocation
Critical	Production and urgent jobs	40%
ETL	Data ingestion and transformation	30%
Analytics	Reporting and queries	20%
Research	Experimental and ML workloads	10%

This prevents experimental workloads from consuming resources required by critical production applications.

Use of Hadoop Concepts

The solution applies YARN Resource Manager, Node Manager, containers, Capacity Scheduler, Fair Scheduler, queues, v Cores, and memory allocation.

This approach provides controlled and fair resource sharing across multiple users.

5. Distributed Storage Backup Restores Outdated Data

Identification of Error

The main design error is the use of replication as if it were a complete backup mechanism, or the use of backups without proper version control and recovery validation.

Replication improves availability, but it does not necessarily preserve an earlier correct version of data. If corrupted or outdated data is replicated, all replicas may contain the same incorrect version.

Cause Analysis

Consider a system where data is replicated asynchronously. A failure occurs before the latest updates are copied to the backup location. During recovery, the system restores an older version because the backup was not synchronized with recent changes.

Another problem occurs when backups overwrite previous copies. If only one backup version exists, it may be impossible to restore the most appropriate version.

Corrective Actions

The improved strategy should include:

1. Use replication for high availability and separate backup mechanisms for data recovery.
2. Create periodic HDFS snapshots.
3. Maintain multiple backup versions instead of overwriting a single backup.
4. Use incremental backups to capture changes between full backups.
5. Define a suitable Recovery Point Objective (RPO).
6. Define a suitable Recovery Time Objective (RTO).
7. Store backup metadata with timestamps and version information.
8. Maintain an independent backup location separate from the primary cluster.
9. Regularly verify backup integrity.
10. Perform recovery testing to ensure that the correct and latest required version can actually be restored.

Use of Hadoop and Distributed Storage Concepts

This solution applies HDFS replication, snapshots, incremental backups, versioning, disaster recovery, RPO, RTO, independent backup storage, and recovery validation.

The final design should follow the principle:

Replication provides availability, while backups provide recovery.